

CEC424

University of Buea — College of Technology (COT)

Introduction to Deep Learning / Full-Stack Web Development

Full-Stack Web Dev

Flask & Django

Flask

Micro · Flexible

+

Django

Batteries-Included



IRIS + RF

Machine Learning API

+

CIFAR-10

CNN Image Classifier

TCHAPGA TCHITO C.

Academic Year 2025/2026 | HTTP · REST · ORM · Templates · ML Deployment

Course Agenda

A Web Fundamentals — HTTP, REST, MVC vs MVT

B Flask Framework — Routing, Templates, Forms

B+ Flask ML API — IRIS + Random Forest

C Django Framework — MVT, ORM, Admin, DRF

C+ Django Image Classifier — CIFAR-10 + CNN

D Database Integration — SQLAlchemy & Django ORM

E REST API Design — DRF vs Flask-RESTX

F Flask vs Django — Comparison & Deployment

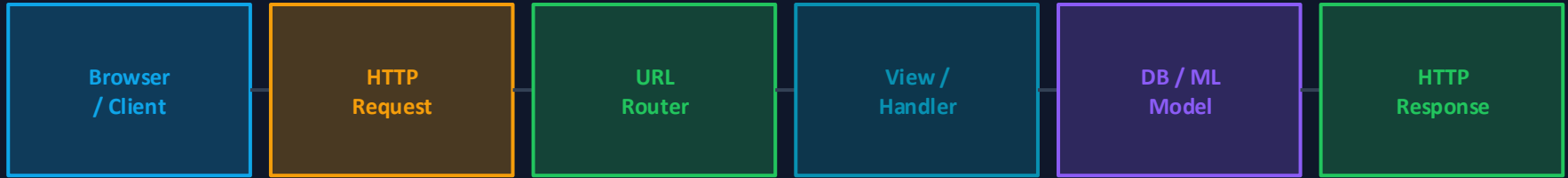


Web Fundamentals

HTTP · REST · MVC vs MVT · Status Codes

HTTP Request-Response Cycle

Every web interaction is a request followed by a response



Method	Meaning	Example	Idempotent	Body	Status
GET	Read	GET /api/flowers	Yes	No	200
POST	Create	POST /api/predict	No	Yes	201
PUT	Replace	PUT /api/flowers/1	Yes	Yes	200
PATCH	Update	PATCH /api/flowers/1	Yes	Yes	200
DELETE	Remove	DELETE /api/flowers/1	Yes	No	204

REST API & Status Codes

Stateless, resource-oriented — JSON over HTTP



```
# — REST API response structure —————
{
  "status": "success",
  "data": {
    "class_name": "Iris-setosa",
    "confidence": 0.97,
    "features": [5.1, 3.5, 1.4, 0.2]
  },
  "meta": {
    "model": "RandomForest v1.0",
    "response_ms": 12
  }
}
```



```
# — Flask REST endpoint —————
@app.route("/api/predict", methods=["POST"])
def api_predict():
    data = request.get_json() # parse body
    if not data or "features" not in data:
        return jsonify({"error": "bad request"}), 400
    pred = model.predict([data["features"]])[0]
    return jsonify({
        "class_name": CLASSES[pred],
        "confidence": float(proba[pred])
    }), 200
```

2xx

Success — 200 OK · 201 Created · 204 No Content

3xx

Redirect — 301 Moved · 302 Found · 304 Not Modified

4xx

Client Error — 400 Bad · 401 Unauth · 403 Forbidden · 404 Not Found · 422 Unprocessable

5xx

Server Error — 500 Internal · 502 Bad Gateway · 503 Unavailable

Flask — Core Concepts

"Micro" means minimal core — you add only what you need

```
from flask import Flask, request, render_template, jsonify

app = Flask(__name__)

# — Basic routes —————
@app.route("/")
def index():
    return render_template("index.html", title="Home")

@app.route("/about")
def about():
    return "<h1>About</h1>"

# — Dynamic URL parameters —————
@app.route("/user/<username>")
def user_profile(username):
    return f"Hello, {username}!"
```

Routing

URL decorated functions.

@app.route('/path', methods=['GET', 'POST'])

Dynamic: /user/<username>

```
# — Handling GET + POST —————
@app.route("/predict", methods=["GET", "POST"])
def predict():
    if request.method == "POST":
        sl = float(request.form["sepal_length"])
        sw = float(request.form["sepal_width"])
        pl = float(request.form["petal_length"])
        pw = float(request.form["petal_width"])
        pred = model.predict([[sl, sw, pl, pw]])[0]
        return render_template("result.html",
                               class_name=CLASSES[pred])
    return render_template("predict.html")

# — Run the app —————
if __name__ == "__main__":
    app.run(debug=True, port=5050, host="0.0.0.0")
```

Templates

Jinja2 engine.

{{ var }} — output variable

{% if %}/{% for %} — logic

{% extends 'base.html' %} — inherit

Request / Response

request.form — POST form data

request.args — URL query params

request.get_json() — JSON body

jsonify() / render_template()

Flask — Jinja2 Templates & Project Structure

```

{# templates/base.html - Jinja2 layout #}
<!DOCTYPE html>
<html>
<head><title>{{ title }}</title></head>
<body>
  <nav>
    <a href="{{ url_for('index') }}">Home</a>
    <a href="{{ url_for('predict') }}">Predict</a>
  </nav>
  {% block content %}{% endblock %}
</body>
</html>

```

```

{# templates/predict.html - extends base #}
{% extends "base.html" %}

```

```

# — Flask-SQLAlchemy - Database ORM —————
from flask_sqlalchemy import SQLAlchemy

app.config["SQLALCHEMY_DATABASE_URI"] = "sqlite:///iris.db"
db = SQLAlchemy(app)

class PredictionLog(db.Model):
    id          = db.Column(db.Integer, primary_key=True)
    class_name  = db.Column(db.String(50))
    confidence  = db.Column(db.Float)
    timestamp   = db.Column(db.DateTime, default=db.func.now())

# Create tables
with app.app_context():
    db.create_all()

```

Flask Project Structure

```

flask_iris_app/
├── app.py           # routes + factory
├── model/iris_rf.pkl # saved ML model
├── templates/       # Jinja2 HTML
│   ├── base.html
│   ├── index.html
│   └── result.html
└── static/          # CSS, JS

```

```

# — Run options —————
# Development
flask run --port 5050 --debug

# Production (Gunicorn)
gunicorn -w 4 -b 0.0.0.0:5050 app:app

# Docker
FROM python:3.11-slim
COPY --from=flask_iris_app . /app
WORKDIR /app
RUN pip install -r requirements.txt

```



Flask Framework

Micro · Explicit · Extensible · Perfect for ML APIs



Flask ML API

IRIS Flower Classification · Random Forest · REST

IRIS Dataset + Random Forest — Overview

150 samples · 4 features · 3 classes · scikit-learn

Feature	Range (cm)	Importance
Sepal Length	4.3 – 7.9	Low (0.09)
Sepal Width	2.0 – 4.4	Lowest (0.03)
Petal Length	1.0 – 6.9	★★★ High (0.44)
Petal Width	0.1 – 2.5	★★★ High (0.43)

Random Forest

Ensemble of N decision trees.
Each tree trains on bootstrap sample.
Final prediction: majority vote.
n_estimators=100 is a solid default.

Model Serving

1. Train model (offline)
2. `joblib.dump(rf, 'model.pkl')`
3. Load once at app startup
4. `rf.predict([[features]])` per request

API Design

POST /api/predict
Body: {features:[sl,sw,pl,pw]}
Returns: {class_name, confidence,
probabilities: {...}}

Flask IRIS API — Complete Implementation

Train → Save → Serve → Test

```
● ● ●
# — Step 1: Train & Save —————
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
import joblib

iris = load_iris()
X_tr, X_te, y_tr, y_te = train_test_split(
    iris.data, iris.target, test_size=0.2,
    random_state=42, stratify=iris.target)

rf = RandomForestClassifier(n_estimators=100,
                           random_state=42)
rf.fit(X_tr, y_tr)
print(f"Accuracy: {rf.score(X_te, y_te):.4f}")

# Save model
joblib.dump(rf, "model/iris_rf_model.pkl")

# — Step 2: Load in Flask app —————
# app.py (at module level – runs ONCE)
model = joblib.load("model/iris_rf_model.pkl")
CLASSES = ["Iris-setosa", "Iris-versicolor", "Iris-virginica"]
```

```
● ● ●
# — Step 3: REST endpoint —————
@app.route("/api/predict", methods=["POST"])
def api_predict():
    data = request.get_json()
    if not data or "features" not in data:
        return jsonify({"error": "bad request"}), 400
    arr = np.array([data["features"]])
    pred = int(model.predict(arr)[0])
    proba = model.predict_proba(arr)[0].tolist()
    return jsonify({
        "class_id": pred,
        "class_name": CLASSES[pred],
        "confidence": round(proba[pred], 4),
        "probabilities": dict(zip(CLASSES,
                                [round(p,4) for p in proba]))
    })

# — Step 4: Test with Python requests —————
import requests
r = requests.post("http://localhost:5050/api/predict",
    json={"features": [5.1, 3.5, 1.4, 0.2]})
print(r.json())
# {"class_name": "Iris-setosa", "confidence": 1.0, ...}
```



Django Framework

MVT · ORM · Admin · DRF · Full-stack batteries included

Django — Architecture & Project Structure

Model-View-Template (MVT) — the "batteries included" philosophy



```
myproject/
├── manage.py
├── myproject/
│   ├── settings.py # project config
│   ├── urls.py     # INSTALLED_APPS, DB, MEDIA
│   └── wsgi.py     # root URL patterns
├── classifier/
│   ├── models.py  # Django app
│   ├── views.py   # ORM models (DB tables)
│   ├── urls.py    # request handlers
│   ├── forms.py   # app URL patterns
│   ├── serializers.py # Django forms
│   ├── admin.py   # DRF serializers
│   └── templates/ # Admin registration
│       └── classifier/
```

```
# — Django batteries included —————
# 1. ORM — no raw SQL needed
# python manage.py makemigrations
# python manage.py migrate

# 2. Admin — zero config
# python manage.py createsuperuser
# → /admin/ auto-generates CRUD for all models

# 3. Auth — built-in
# @login_required decorator
# User, Group, Permission models

# 4. Django REST Framework (DRF)
# pip install djangorestframework
# → ViewSets, Serializers, Routers
# →Browsable API at /api/

# 5. Run server
# python manage.py runserver 8080
```

Django ORM — Models, Queries & Migrations

Write Python objects, not SQL — Django translates for you

```
● ● ●
# — models.py —————
from django.db import models

class PredictionLog(models.Model):
    timestamp = models.DateTimeField(auto_now_add=True)
    predicted_class = models.CharField(max_length=50)
    confidence = models.FloatField()
    image_filename = models.CharField(max_length=200)

    class Meta:
        ordering = ["-timestamp"]

    def __str__(self):
        return f"{self.predicted_class} ({self.confidence:.0%})"

# — admin.py — register to expose in /admin/ —————
```

```
● ● ●
# — QuerySet API — no SQL needed —————
# SELECT all
logs = PredictionLog.objects.all()

# WHERE
high_conf = PredictionLog.objects.filter(
    confidence__gte=0.95)

# ORDER + LIMIT
recent = PredictionLog.objects.order_by(
    "-timestamp")[:10]

# CREATE
PredictionLog.objects.create(
    predicted_class="airplane",
    confidence=0.97,
    image_filename="test.jpg")
```

Django Migrations — Key Commands

```
● ● ●
python manage.py makemigrations # detect model changes → create migration files
python manage.py migrate       # apply migrations to database
python manage.py showmigrations # list all migrations + status
python manage.py sqlmigrate app 0001 # show SQL for a migration
```

Django Views & Django REST Framework

FBV, CBV, and DRF API views



```
# — Function-Based View (FBV) —————
from django.shortcuts import render
from django.http import JsonResponse

def index(request):
    return render(request, "classifier/index.html")

def classify(request):
    if request.method == "POST":
        image = request.FILES.get("image")
        pred, conf = run_cnn(image)
        return render(request, "classifier/result.html",
                      {"class_name": pred, "confidence":
conf})
    return render(request, "classifier/index.html")

# — Class-Based View (CBV) —————
from django.views import View

class ClassifyView(View):
    def get(self, request):
        return render(request, "classifier/index.html")

    def post(self, request):
        image = request.FILES.get("image")
        pred, conf = run_cnn(image)
        return render(request, "result.html",
                      {"class": pred})
```



```
# — Django REST Framework (DRF) —————
from rest_framework.decorators import api_view
from rest_framework.response import Response
from rest_framework import status

@api_view(["POST"])
def api_classify(request):
    if "image" not in request.FILES:
        return Response({"error": "no image"},
                        status=status.HTTP_400_BAD_REQUEST)
    image = request.FILES["image"]
    pred, conf = run_cnn(image)
    return Response({
        "class_name": pred,
        "confidence": conf,
    }, status=status.HTTP_200_OK)

# — urls.py —————
from django.urls import path
from . import views

urlpatterns = [
    path("", views.index, name="index"),
    path("classify/", views.classify, name="classify"),
    path("api/classify/", views.api_classify, name="api"),
]
```



Django CNN Classifier

CIFAR-10 Image Upload · CNN Model · Prediction Logging

CIFAR-10 + CNN — Overview & Architecture

32×32 RGB images · 10 classes · Keras CNN · Django deployment

0: airplane

1: automobile

2: bird

3: cat

4: deer

5: dog

6: frog

7: horse

8: ship

9: truck

```
# CNN Architecture (Keras functional API)
inputs = keras.Input(shape=(32,32,3))
x = data_augmentation(inputs) # RandomFlip, RandomRotation, RandomZoom

# Block 1 — 32 filters
x = Conv2D(32, (3,3),padding="same")(x); x = BatchNormalization()(x); x = Activation("relu")(x)
x = Conv2D(32, (3,3),padding="same")(x); x = BatchNormalization()(x); x = Activation("relu")(x)
x = MaxPooling2D(2,2)(x); x = Dropout(0.25)(x)

# Block 2 — 64 filters
x = Conv2D(64, (3,3),padding="same")(x); x = BatchNormalization()(x); x = Activation("relu")(x)
x = Conv2D(64, (3,3),padding="same")(x); x = BatchNormalization()(x); x = Activation("relu")(x)
x = MaxPooling2D(2,2)(x); x = Dropout(0.30)(x)

# Block 3 — 128 filters
x = Conv2D(128, (3,3),padding="same")(x); x = BatchNormalization()(x); x = Activation("relu")(x)
x = MaxPooling2D(2,2)(x); x = Dropout(0.35)(x)

# Head
x = Flatten()(x); x = Dense(256,activation="relu")(x); x = Dropout(0.45)(x)
outputs = Dense(10,activation="softmax")(x)
```

Django Image Upload + CNN Inference

Full pipeline: upload form → preprocess → predict → log → display

```
● ● ●
# — views.py — Django image prediction —
import numpy as np
import tensorflow as tf
from PIL import Image
from django.shortcuts import render

_model = None
CLASSES = ["airplane", "automobile", "bird", "cat", "deer",
            "dog", "frog", "horse", "ship", "truck"]

def get_model():
    global _model
    if _model is None:
        from django.conf import settings
        _model = tf.keras.models.load_model(
            settings.ML_MODEL_PATH)
    return _model
```

```
def preprocess_image(img_file):
```

```
● ● ●
{# templates/classifier/index.html — upload form #}
<form method="post" action="/classify/" enctype="multipart/form-data">
    {% csrf_token %}
    <input type="file" name="image" accept="image/*" required>
    <button type="submit">🐶 Classify Image</button>
</form>

{# result.html — probability bars #}
{% for cls, prob in probabilities %}
    <div style="width:{{ prob }}%">{{ cls }}
{% endfor %}
```

```
● ● ●
def classify(request):
    if request.method == "POST":
        image = request.FILES["image"]
        model = get_model()
        arr = preprocess_image(image)
        preds = model.predict(arr, verbose=0)[0]
        pred_i = int(np.argmax(preds))
        context = {
            "class_name": CLASSES[pred_i],
            "confidence": round(float(preds[pred_i])*100, 2),
            "probabilities": sorted(
                zip(CLASSES, [round(float(p)*100,2) for p in
                    preds]),
                key=lambda x:-x[1]),
            "filename": image.name,
        }
        # Log to DB
        from .models import PredictionLog
        PredictionLog.objects.create(
```



Database Integration

SQLAlchemy (Flask) · Django ORM · SQLite → PostgreSQL

Flask-SQLAlchemy vs Django ORM

Two ORM approaches — same concept, different API

```

# — Flask + SQLAlchemy —————
from flask_sqlalchemy import SQLAlchemy

app.config["SQLALCHEMY_DATABASE_URI"] = (
    "sqlite:///prod.db"          # development
    # "postgresql://user:pw@host/db" # production
)
db = SQLAlchemy(app)

class Flower(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    class_name = db.Column(db.String(50), nullable=False)
    confidence = db.Column(db.Float)
    created_at = db.Column(db.DateTime, default=db.func.now())

# Create tables (once)
with app.app_context():
    db.create_all()

# CRUD
# Create
f = Flower(class_name="Setosa", confidence=0.97)
db.session.add(f); db.session.commit()

# Read
all_f = Flower.query.all()
setosa = Flower.query.filter_by(class_name="Setosa").all()

# Update
f.confidence = 0.99; db.session.commit()

# Delete
db.session.delete(f); db.session.commit()
```

```

# — Django ORM —————
# settings.py — switch DB with one line
DATABASES = {
    "default": {
        # Development
        "ENGINE": "django.db.backends.sqlite3",
        "NAME": BASE_DIR / "db.sqlite3",
        # Production (PostgreSQL)
        # "ENGINE": "django.db.backends.postgresql",
        # "NAME": "mydb", "USER": "user",
        # "PASSWORD": "pw", "HOST": "localhost",
    }
}

# models.py
class Flower(models.Model):
    class_name = models.CharField(max_length=50)
    confidence = models.FloatField()
    created_at = models.DateTimeField(auto_now_add=True)

# python manage.py makemigrations && migrate

# CRUD
# Create
Flower.objects.create(class_name="Setosa", confidence=0.97)
# Read
Flower.objects.filter(class_name="Setosa").order_by("-confidence")

# Update
Flower.objects.filter(id=1).update(confidence=0.99)
```



REST API Design

DRF ViewSets · Serializers · Flask-RESTX · Versioning

Django REST Framework — Serializers & ViewSets

The fastest path to a production-quality REST API



```
# — serializers.py —————
from rest_framework import serializers
from .models import PredictionLog

class PredictionLogSerializer(serializers.ModelSerializer):
    class Meta:
        model = PredictionLog
        fields = ["id", "timestamp",
                  "predicted_class", "confidence"]
        read_only_fields = ["id", "timestamp"]

# — Input serializer for validation —————
class PredictSerializer(serializers.Serializer):
    features = serializers.ListField(
        child=serializers.FloatField(), min_length=4,
        max_length=4)

    def validate_features(self, value):
        if any(v < 0 for v in value):
            raise serializers.ValidationError(
                "Features must be non-negative")
        return value
```



```
# — views.py — ModelViewSet —————
from rest_framework import viewsets, permissions
from .serializers import PredictionLogSerializer

class PredictionLogViewSet(viewsets.ModelViewSet):
    queryset = PredictionLog.objects.all()
    serializer_class = PredictionLogSerializer
    permission_classes = [permissions.IsAuthenticated]

# — urls.py — Router auto-generates all endpoints —
from rest_framework.routers import DefaultRouter
from . import views

router = DefaultRouter()
router.register(r"logs", views.PredictionLogViewSet)

urlpatterns = [
    path("api/", include(router.urls)),
    # Auto-creates:
    # GET    /api/logs/           → list
    # POST   /api/logs/           → create
    # GET    /api/logs/{id}/     → retrieve
    # PUT    /api/logs/{id}/     → update
    # DELETE /api/logs/{id}/     → destroy
]
```



Flask vs Django

Comparison · When to use which · Deployment

Flask vs Django — Full Comparison

Choose the right tool for the job

Aspect	Flask	Django
Philosophy	Micro — minimal core, add extensions	Batteries included — everything built-in
Learning Curve	Lower — 30 min to first route	Higher — MVT + ORM + migrations
ORM	SQLAlchemy (extension)	Built-in Django ORM
Admin Panel	Flask-Admin (extension)	Built-in /admin/ — zero config
Auth System	Flask-Login (extension)	Built-in User / Group / Permission
REST API	Flask-RESTX / flask-smorest	Django REST Framework (DRF)
Templates	Jinja2	Django Template Language (Jinja2 option)
ML Serving	Excellent — lightweight, fast startup	Good — more overhead but full DB
Best for	APIs, microservices, ML serving	Full web apps, CMS, e-commerce
Run command	flask run --port 5050	python manage.py runserver 8080

Deployment — Both Frameworks

Gunicorn + Nginx is the production standard for both



```
# — Flask Docker deployment —————
# Dockerfile
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["gunicorn", "--workers", "4",
     "--bind", "0.0.0.0:5000", "app:app"]
```

```
# requirements.txt
flask==3.0.3
flask-sqlalchemy==3.1.1
psycopg2-binary==2.9.9
```



```
# — Django Docker deployment —————
# Dockerfile
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
RUN python manage.py collectstatic --noinput
EXPOSE 8000
CMD ["gunicorn", "--workers", "4",
     "--bind", "0.0.0.0:8000", "myproject.wsgi:application"]
```

```
# requirements.txt
django==5.0.6
django-rest-framework==3.15.1
```



```
# — docker-compose.yml — web + db —————
services:
  web:
    build: .
    ports: ["8000:8000"]
    environment: [DATABASE_URL=postgres://user:pw@db/mydb]
    depends_on: [db]
```

Heroku / Railway

Free tier available.
git push heroku main
Auto-detects Procfile.

DigitalOcean App

Managed platform.
Connect GitHub repo.
Auto-deploy on push.

AWS EC2 + Nginx

Full control.
gunicorn + nginx config.
Best for production ML.

Google Cloud Run

Serverless containers.
Auto-scales to zero.
Pay per request.

Key Takeaways

HTTP & REST

Every web action = HTTP request+response. REST: stateless, resource-oriented, JSON over HTTP.

Flask

Micro-framework — `@app.route()`, `render_template()`, `jsonify()`. Ideal for ML APIs.

IRIS + RF

`joblib.dump/load` to persist model. Load once at startup. `POST /api/predict` → JSON.

Django

MVT: `urls.py` → `views.py` → `models.py` → `templates`. `python manage.py runserver`.

CIFAR-10 CNN

`request.FILES['image']` → PIL resize → numpy → `model.predict()` → jsonify probabilities.

ORM

SQLAlchemy (Flask) or Django ORM — Python objects → SQL. No raw SQL needed.

DRF

`ModelViewSet` + `DefaultRouter` auto-generates 5 endpoints. Serializer validates input.

Deploy

Gunicorn + Docker → Heroku/Railway/Cloud Run. WSGI for Flask & Django.